

SHADOWGRAM

Blueprint & Simple Explanation

Behavioral Graph Detection of Coordinated AI-Fraud Accounts

A guide to **understanding ShadowGram yourself** and to **explaining it confidently to a judge, teammate, or anyone who asks “What exactly are you building?”** The language is kept simple first, then the actual technical pipeline is broken down. Sources: the Idea Proposal, HackAthena submission, Project Abstract, and research extracts.

SHADOWGRAM — Simple Explanation

One-line explanation

ShadowGram is a fraud-detection system that looks at how multiple accounts behave and finds hidden groups of accounts that are likely being operated together. It then explains exactly why those accounts were connected.

The important idea: **we are not trying to prove that an account is fake.** Instead, we ask:

“Do these accounts behave so similarly that it looks like they are being controlled or operated together?”

This is the main idea behind ShadowGram.

1. What problem are we solving?

Imagine an attacker wants to commit fraud. Instead of using one account, they create **20, 50, or 100 different accounts**. Each account can have a different username, profile, bio, slightly different messages, and different-looking content.

So if a traditional fraud system checks **one account at a time**, each account might look normal:

Account A → Looks normal
Account B → Looks normal
Account C → Looks normal
Account D → Looks normal

But when we look at their **behavior together**, we may notice:

A —
B —
C — All behave almost identically
D —
E —

- they become active at almost the same time
- they follow the same navigation sequence
- they post at similar intervals
- their typing patterns are similar
- their content has high semantic similarity
- their client/environment signals match

That combination is much more interesting than any one signal by itself. This is the **relational approach** of ShadowGram.

2. The easiest way to understand ShadowGram

Think of ShadowGram like a **detective**. Suppose five people appear to be strangers. A normal system asks: *“Does Person A look suspicious?”* ShadowGram asks: *“Do A, B, C, D and E behave as if they are working together?”* Then it collects evidence.

```
Account A
  ↓
Typing pattern
Navigation
Timing
Content
Environment
  ↓
Behavioral Fingerprint
```

We do the same for every account. Then we calculate how similar their behaviors are:

```
Account A ↔ Account B
Account A ↔ Account C
Account B ↔ Account C
...
```

Then we turn those relationships into a **graph**. Finally, we find groups/clusters in that graph.

3. The five behavioral fingerprints

This is one of the most important things to remember when explaining ShadowGram. We use **five types of evidence**.

1. Interaction / Typing

We look at how a user interacts with the application: typing speed, keystroke timing, hesitation, mouse/pointer movement, click intervals, and scrolling. The newer submission describes more specific measures such as **Keystroke Flight Time, Dwell Time variance, and pointer curvature/jerk**.

2. Navigation

We observe the order in which the account moves through the application:

```
Home
  ↓
Search
  ↓
Profile
  ↓
Post
```

If many accounts repeatedly follow the same unusual route, that can become evidence of coordination. The project models these routes as **finite-state-machine paths**.

3. Timing

We examine **when actions happen**.

```
Account A → posts at 10:00:01
Account B → posts at 10:00:02
Account C → posts at 10:00:01
Account D → posts at 10:00:02
```

One coincidence doesn't mean much, but repeated synchronization across multiple actions can be meaningful. The submission uses micro-temporal synchronization and gives a prototype threshold of **$\Delta t < 1.4$ seconds**.

4. Content

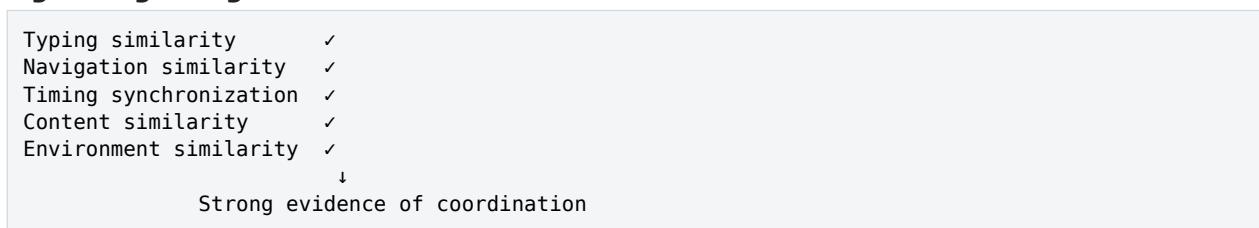
Two accounts might not use exactly the same sentence. For example, “Click here to claim your reward.” and “Get your reward by clicking this link.” The words are different, but the **meaning is similar**. ShadowGram uses **Sentence Transformers**, specifically all-MiniLM-L6-v2 in the submission, to convert text into numerical vectors and compare their semantic similarity.

5. Environment

In the **simulated consenting ShadowGram environment**, we can collect environment-related signals such as browser, OS, screen resolution, timezone, language, Canvas/WebGL information, and device/network identifiers. This is explicitly described as **prototype-only** and consent-based rather than something collected from scraped real accounts.

4. Why do we need five signals?

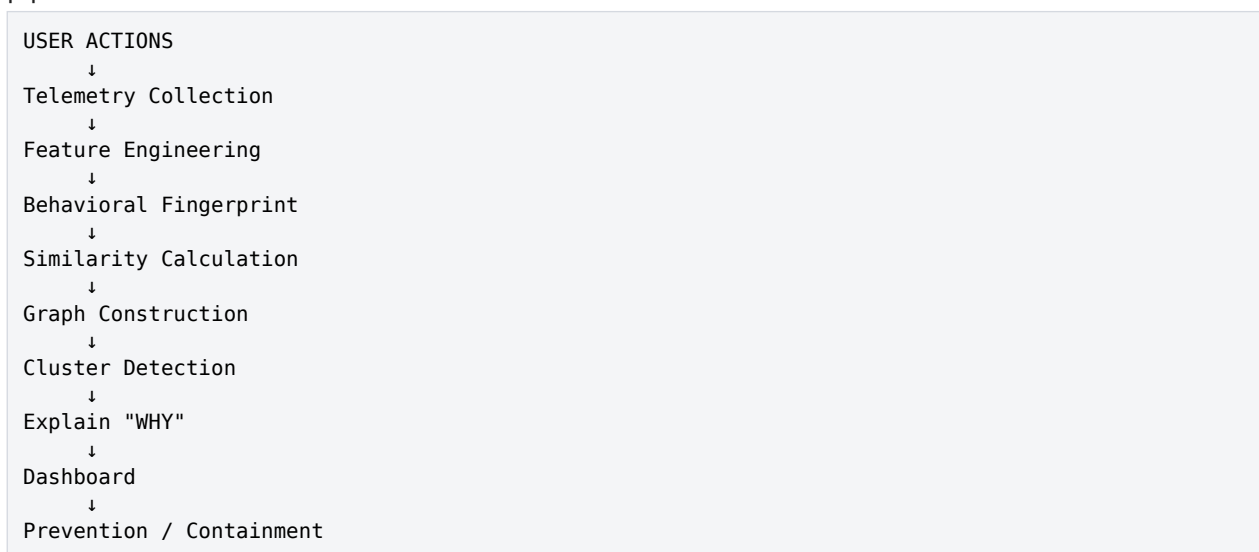
This is extremely important when someone asks: “*What if two normal people happen to type similarly?*” Exactly. Two unrelated people could have similar typing speed, similar working hours, and similar browsing behavior — so **one signal is not enough**. ShadowGram looks for **multiple signals agreeing at the same time**.



The proposal explicitly says that coordination should be based on convergence across multiple independent signals rather than one suspicious feature.

5. What actually happens technically?

Now let's move from the simple explanation to the **actual implementation**. The complete pipeline is:



The project abstract divides this into four main processing stages.

STEP 1 — Create the simulated environment

First, we need somewhere where our accounts can interact. We create a **simulated web/social-platform environment**. For the MVP, the proposal suggests starting small:

```
5 normal accounts
3 coordinated attacker accounts
2 unrelated suspicious accounts
```

Then we can scale the demonstration to around **100 accounts**.

Technologies (likely stack): HTML/CSS/JavaScript, React / Next.js, browser-based event tracking.

Technical concept — Client-side telemetry: we observe application events happening in the browser.

STEP 2 — Collect user behavior

We add JavaScript event listeners to capture interactions, such as:

```
keydown
pointerdown
click
scroll
route change
```

The submission describes lightweight JavaScript event hooks and throttling telemetry to **50 ms intervals** to avoid locking the browser thread. Instead of storing “User clicked something,” we store useful behavioral information:

```
timestamp
action
page
keyboard timing
mouse information
```

Technology: JavaScript Event Listeners. **Concept — Telemetry collection:** collecting information about what is happening in the system.

STEP 3 — Send telemetry to the backend

The browser needs to send this information to our backend. The submitted architecture uses **FastAPI** to receive the behavioral data.

```
Browser
↓
POST /telemetry
↓
FastAPI
↓
Process data
```

The abstract specifies **REST/WebSocket endpoints** for streaming telemetry to the backend.

- **Technologies:** Python, FastAPI, REST API, WebSockets
- **API:** allows the frontend and backend to communicate.
- **WebSocket:** useful when information must move continuously / near-real-time between browser and server.

STEP 4 — Convert raw behavior into features

Raw events are not directly useful to the ML system. Suppose we have:

```
keydown at 10:01:01.200
keydown at 10:01:01.450
keydown at 10:01:01.700
```

We can convert that into *Typing speed* = X , *Average delay* = Y , *Typing variance* = Z . Similarly, raw navigation events become a navigation feature (Home → Search → Profile → Post), and raw timestamps become action intervals and timing features. This is called **Feature Engineering**.

The abstract describes converting numerical metrics into normalized vectors: $x \in R^D$. In simple words: we convert human/account behavior into numbers that the algorithms can compare.

STEP 5 — Create the behavioral fingerprint

Now each account gets its own numerical representation. Something like:

```
Account A

Typing speed      → 4.2
Hesitation        → 1.8
Mouse speed       → 0.73
Navigation score  → 0.91
Active start      → 20.5
Active end        → 23.1
Post interval     → 47
Content similarity → 0.88
```

The original proposal gives a similar example of behavioral vectors for two accounts. This collection of values is the account's **Behavioral Fingerprint** — think of it like a behavioral signature.

STEP 6 — Detect unusual individual behavior

The proposed first version uses **Isolation Forest**, a classical machine-learning algorithm for **anomaly detection**. It asks: “Does this account behave unusually compared with the other accounts?”

```
Most accounts:
9 AM — 6 PM

Account X:
2 AM — 5 AM
```

Account X could become an anomaly. But remember: **Anomaly ≠ fraud**. An unusual user can simply be a genuine unusual user. That's why ShadowGram doesn't stop here. The proposal specifically uses Isolation Forest for unusual individual behavioral patterns.

Technology: scikit-learn. **Concept:** Anomaly Detection.

STEP 7 — Compare accounts

Now we compare Account A with Account B using **Cosine Similarity**. Imagine each account is represented as a vector:

```
A = [0.8, 0.7, 0.9, 0.8]
B = [0.81, 0.71, 0.88, 0.79]
```

Cosine similarity tells us how similar their vectors are:

```
Similarity ≈ 1
    ↓
Very similar

Similarity ≈ 0
    ↓
```

Very different

The proposal specifically identifies **cosine similarity** for behavioral-vector comparison.

STEP 8 — Compare the actual content

This is where **Sentence Transformers** comes in. Suppose:

Account A:
"Win ₹500 by clicking this link."

Account B:
"Click the link and receive a ₹500 reward."

Traditional word matching might see differences. A sentence embedding model converts each sentence into a numerical vector. The submission uses all-MiniLM-L6-v2, which produces a **384-dimensional embedding** according to the project abstract.

Sentence A → [0.12, 0.42, ...]
Sentence B → [0.14, 0.40, ...]
 ↓
 Cosine similarity

If the semantic similarity is high, it becomes another piece of evidence.

- **Technologies:** Hugging Face Transformers, Sentence Transformers, all-MiniLM-L6-v2
- **Concepts:** NLP, embeddings, vector representation, semantic similarity, cosine similarity

STEP 9 — Combine all the signals

Now we have multiple similarities:

Typing similarity = 0.90
Navigation similarity = 0.95
Timing similarity = 0.93
Content similarity = 0.89
Environment similarity = 0.91

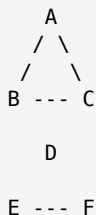
We combine them into a **composite similarity score**. The project abstract represents this concept as:

$$S_{comp}(u,v) = \sum w_k S_k(u,v)$$

In simple English: each behavioral signal contributes to the final similarity score, with different weights. The submission specifies a prototype threshold of $S_{comp} \geq 0.78$ across at least three categories before creating an edge.

STEP 10 — Build the graph

This is where the name **Behavioral Graph Detection** becomes important. Imagine:



Each circle is an **account**. Each line is a **behavioral relationship**. The stronger the relationship, the stronger the edge.

```
A — B
0.91

B — C
0.88

A — C
0.94
```

This creates a **weighted graph**. **Technology: NetworkX** — the Python library used to construct and analyze the graph in the submitted architecture. **Concept: Graph theory**.

```
Node = Account
Edge = Behavioral relationship
Edge weight = Strength of relationship
```

STEP 11 — Find suspicious groups

Now we have a graph containing potentially hundreds of accounts. We need to identify groups that are strongly connected. The submission proposes **Louvain Community Detection**, which detects communities/groups in a graph by optimizing **modularity**. You don't need to memorize the mathematical equation — just remember:

Louvain looks at the graph and finds groups of accounts that are much more strongly connected to each other than to the rest of the network.

The project abstract uses Louvain community detection for isolating coordinated clusters. The earlier proposal also mentions **DBSCAN/HDBSCAN** as possible clustering methods. If someone asks about the final graph-clustering method, the **HackAthena submission/abstract currently specifies NetworkX + Louvain**.

STEP 12 — Create the “WHY” explanation

This is probably the **most important feature of ShadowGram**. Most fraud systems might say “△ Risk Score: 94%” — but the reviewer asks: **WHY?** ShadowGram answers:

- 94% timing similarity
- identical 4-step navigation pattern
- 0.89 semantic similarity
- similar interaction cadence
- matching environment signals

This is the **Explainability Layer / Why Layer**. The submitted concept calls this an inspectable evidence dossier. Instead of “94% suspicious,” we give:

WHY ARE THESE ACCOUNTS LINKED?

- ✓ Timing synchronization
- ✓ Navigation similarity
- ✓ Typing similarity
- ✓ Content similarity
- ✓ Environment similarity

Evidence:

“Accounts repeatedly performed the same 4-step navigation sequence within similar time intervals.”

That makes the system easier for a human reviewer to understand.

STEP 13 — Show everything in a dashboard

Now we need to present the results. The proposed dashboard contains:

- **Cluster Overview** — e.g. Accounts analyzed: 100 · Shadow clusters: 3 · Cluster score: 0.91
- **Behavioral Fingerprint** — the individual account's behavioral features
- **Compare Behavior** — Account A vs Account B across the fingerprint dimensions
- **Why Are These Accounts Linked?** — shows the evidence
- **Timeline** — Account created → First synchronized activity → Similar posts → Suspicious links shared → Shadow Cluster Detected
- **Prevention** — what action the platform could take

STEP 14 — Prevention / containment

ShadowGram shouldn't just say “This looks suspicious.” It should suggest what the platform can do:

- require additional verification
- restrict suspicious-link distribution
- send the cluster for human review
- monitor newly created accounts showing similar behavior

The submission also mentions **step-up authentication such as WebAuthn/FIDO2** as a possible containment mechanism.

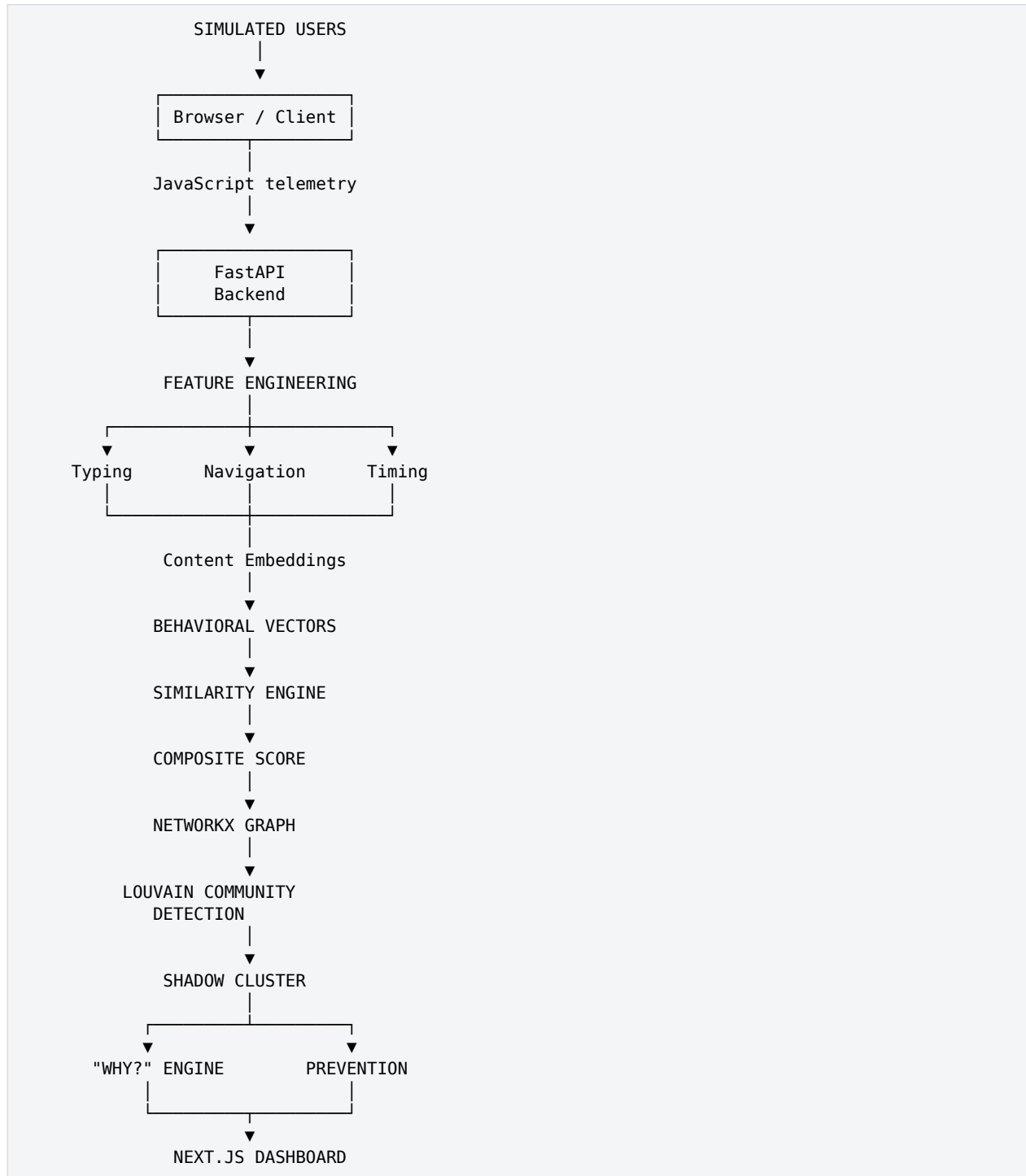
The complete technical stack

The easiest table to memorize (combining the HackAthena submission and the project abstract):

Part	Technology	What it does
Frontend	Next.js / React	User interface
Styling	Tailwind CSS	Dashboard design
Browser telemetry	JavaScript	Collects behavior
Communication	REST / WebSocket	Sends telemetry
Backend	FastAPI + Python	Receives/processes data
ML	scikit-learn	Isolation Forest / anomaly detection
NLP	Sentence Transformers	Understands content similarity
NLP model	all-MiniLM-L6-v2	Generates text embeddings
Similarity	Cosine similarity	Compares vectors
Graph	NetworkX	Builds account graph
Clustering	Louvain	Finds communities
Visualization	React-Force-Graph	Displays graph
Data generation	Python	Creates simulated accounts
Deployment/demo	Local/offline	Runs without external AI APIs

How the data flows

This is the diagram you should understand:



What exactly are we going to build for the hackathon?

The project abstract gives a **48-hour roadmap**.

Phase 1 — Hours 0-16

Build: client telemetry (Browser → JavaScript event listeners → Capture behavior), the FastAPI backend with REST/WebSocket endpoints, and synthetic data (simulated accounts).

Concepts you learn/use: JavaScript events, APIs, WebSockets, Python, FastAPI, synthetic data.

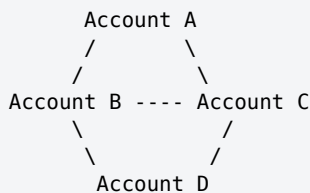
Phase 2 — Hours 16-32

This is the **ML + graph intelligence part**. Build: feature engineering (raw behavior → numerical features), Sentence Transformers (text → embeddings), cosine similarity, NetworkX graph, and Louvain communities.

```
Raw data
↓
Features
↓
Similarity
↓
Graph
↓
Clusters
```

Phase 3 — Hours 32-42

Now build the actual visual product — the **Next.js dashboard** showing Accounts, Clusters, Graph, Similarity, Evidence, Timeline, and Prevention, plus **React-Force-Graph** to display the network visually. The suspicious cluster can visually stand out.



Phase 4 — Hours 42-48

This is the **demo/rehearsal stage**. The submitted demo flow is:

- 1 Start 3 automated AI/bot accounts
- 2 They perform coordinated actions
- 3 ShadowGram receives telemetry
- 4 Behavioral similarities increase
- 5 Graph edges appear
- 6 Cluster emerges
- 7 Open the "WHY?" panel
- 8 Evidence is displayed
- 9 Trigger containment/quarantine

That gives you a complete **end-to-end story** rather than just showing an ML model.

What should YOU remember for explaining the project?

You don't need to remember every equation. Remember these **7 things**:

1. Problem

Fraudsters can operate many accounts together, while each individual account may look normal.

2. Our idea

Instead of checking accounts individually, we compare their behavior and look for coordination.

3. Five signals

Typing + Navigation + Timing + Content + Environment

4. Machine learning

- **Isolation Forest** detects unusual behavior.
- **Sentence Transformers** understand similarity between content.
- **Cosine similarity** compares behavioral/content vectors.

5. Graph

NetworkX creates: $\text{Account} = \text{Node} \cdot \text{Relationship} = \text{Edge} \cdot \text{Similarity} = \text{Edge Weight}$

6. Clustering

Louvain finds groups of strongly connected accounts.

7. Main innovation — the “Why” layer

Instead of saying “These accounts are suspicious,” ShadowGram says “These accounts are linked because their timing, navigation, typing and content behavior show strong similarity.” That’s the part that makes the system explainable.

A 30-second explanation you can memorize

If someone suddenly asks “What is ShadowGram?”, you can say:

ShadowGram is a fraud-detection system that detects coordinated groups of suspicious accounts. Instead of checking whether one account is fake, we look at whether multiple accounts behave as if they are being operated together. We collect five types of behavioral signals: typing and interaction patterns, navigation, timing, content similarity, and environment signals.

We convert these behaviors into numerical features, compare the accounts, and build a graph where accounts are connected based on behavioral similarity. Then we use graph community detection to find suspicious groups.

The main feature is our “Why” layer. Instead of simply saying “this account is 90% suspicious,” ShadowGram explains why the accounts were connected, such as similar timing, navigation, typing, and content.

*So, in one sentence: **ShadowGram turns hidden behavioral coordination between accounts into an explainable fraud graph.***

A slightly more technical explanation for judges

If a judge asks “How does the system technically work?”:

Technically, ShadowGram follows a four-stage pipeline.

First, our client-side JavaScript SDK collects behavioral telemetry such as keyboard events, pointer interactions, navigation routes, and timestamps from the simulated environment. This data is sent to a FastAPI backend through REST or WebSocket endpoints.

Second, we perform feature engineering and normalization to convert the raw telemetry into behavioral feature vectors. User-generated text is converted into 384-dimensional embeddings

using the local *all-MiniLM-L6-v2 Sentence Transformer*, and cosine similarity is used to compare accounts.

Third, we construct a weighted *NetworkX* graph where each account is a node and an edge represents strong behavioral similarity between two accounts. The composite similarity combines multiple behavioral signals, and Louvain community detection is then used to identify strongly connected account communities.

Finally, the *Next.js* dashboard visualizes these clusters using an interactive graph and generates a “Why are these accounts linked?” evidence card explaining the signals responsible for each connection. The system can then recommend containment actions such as additional verification, restricting suspicious links, or human review.

The important point is that *ShadowGram* reports signals consistent with coordinated operation rather than claiming that multiple accounts belong to the same person.

One important distinction to understand

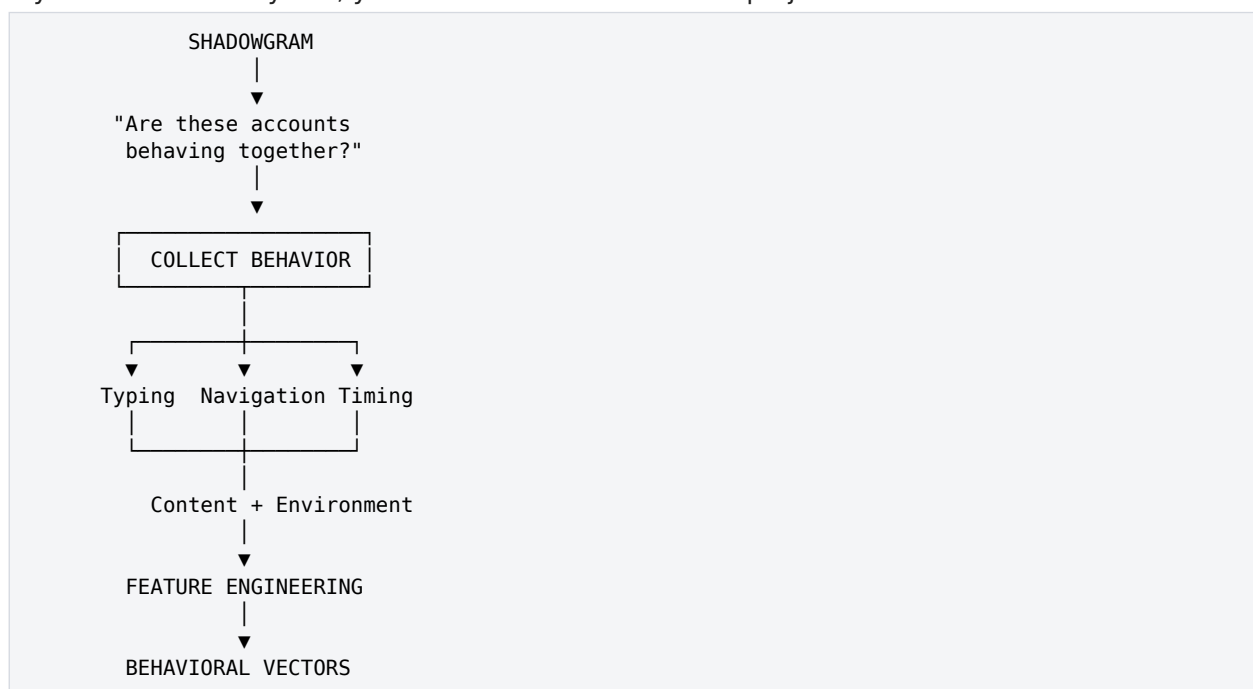
Your documents contain **two slightly different versions of the project framing**. The earlier Idea Proposal describes *ShadowGram* more generally as **Behavioral Graph Detection of Coordinated AI-Fraud Accounts** and discusses options such as Isolation Forest, cosine similarity, DBSCAN/HDBSCAN and *NetworkX*.

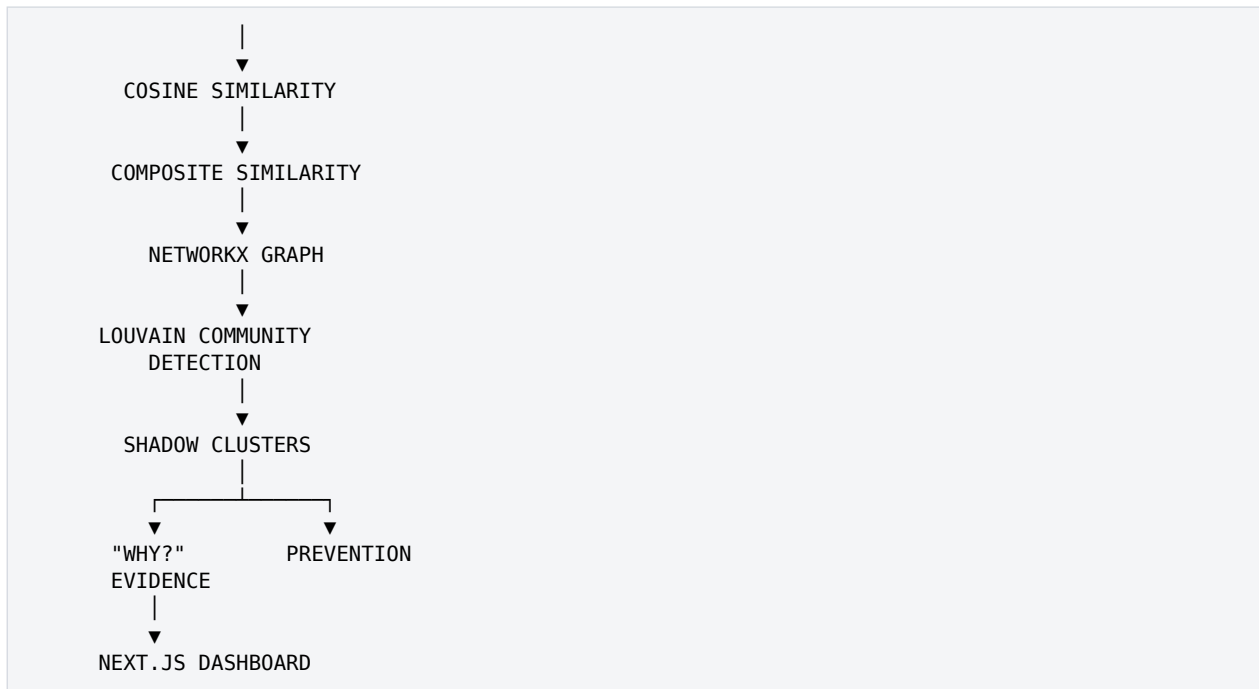
The later *HackAthena* submission/abstract makes the implementation more specific — **Coordinated AI-Agent Swarms & Synthetic Persona Rings** — with FastAPI → Sentence Transformers → *NetworkX* → Louvain → *Next.js/React-Force-Graph*.

So for the **actual HackAthena implementation**, use the **later, more specific architecture as your working blueprint**, while treating the earlier proposal's DBSCAN/HDBSCAN and broader alternatives as conceptual/background options.

The entire project in one picture

If you remember only this, you can reconstruct the whole project:





The core story:

Collect behavior → turn it into numbers → compare accounts → build relationships → find groups → explain the evidence → contain the suspicious group. That is ShadowGram.